

TECHNICAL PROJECT SPECIFICATION

Soul of Nepal

Implementation & Technical Specification

A JSON-first Nepal travel, trekking, adventure, recovery and wellness information platform built around an extremely simple publishing workflow.

Version: 1.0

Development Priority: Functionality & Usability First

Primary Stack: Next.js + Supabase + Cloudinary

Content Architecture: Structured JSON

Document Purpose: Master implementation specification for the vibe-coding agent

Table of Contents

1. Project Vision	16. Cloudinary Architecture
2. Core Development Principles	17. Database Architecture
3. Technology Stack	18. DB.sql Requirement
4. High-Level Architecture	19. Authentication & Authorization
5. Content Architecture	20. Admin Panel
6. Homepage	21. Publishing Workflow
7. Global Search	22. SEO Foundation
8. JSON-First Article System	23. Advertising Architecture
9. Article JSON Schema	24. Security Requirements
10. Supported Content Blocks	25. Environment & Credentials
11. Inline Formatting	26. Recommended Project Structure
12. JSON Importer	27. Phased Execution Plan
13. Preview System	28. Testing & Acceptance Criteria
14. Direct Preview Editing	29. Vibe-Coding Agent Rules
15. Image Workflow	30. Definition of Done

1. Project Vision

Soul of Nepal is a content-first Nepal information platform designed primarily for:

- International travelers
- Trekkers
- Adventure travelers
- Yoga seekers
- Wellness travelers
- People preparing for a trip to Nepal

The website should feel like a **trusted independent Nepal knowledge platform**, not an obvious advertisement for a particular retreat or business.

The platform's main purpose is to provide useful, searchable and well-structured information about Nepal while creating a scalable publishing system.

Fundamental product objective: The administrator should be able to publish a complete article without manually building a webpage or repeatedly creating and formatting content blocks.

2. Core Development Principles

2.1 Functionality Before Visual Design

The first implementation must focus almost entirely on:

- Usability
- Functionality
- Architecture
- Database
- Authentication
- Security
- Article management
- JSON processing
- Image handling
- Search
- Publishing
- SEO foundations

- Advertisement infrastructure

Use a simple skeleton interface during the initial build.

Do not spend significant development time on:

- Final colors
- Final typography
- Complex animations
- Decorative graphics
- Premium visual effects
- Advanced transitions
- Final photography styling

The complete visual identity will be designed after the functional foundation is proven.

2.2 Remove Repetitive CMS Work

The administrator does not want to manually construct articles block-by-block. The entire system must therefore be optimized around:

WRITE ARTICLE → CONVERT TO JSON → PASTE JSON → PREVIEW → ADD IMAGES → PUBLISH

2.3 JSON Is the Source of Truth

The JSON explicitly defines the structure of the article. The website must not guess whether text is a title, heading, paragraph or list.

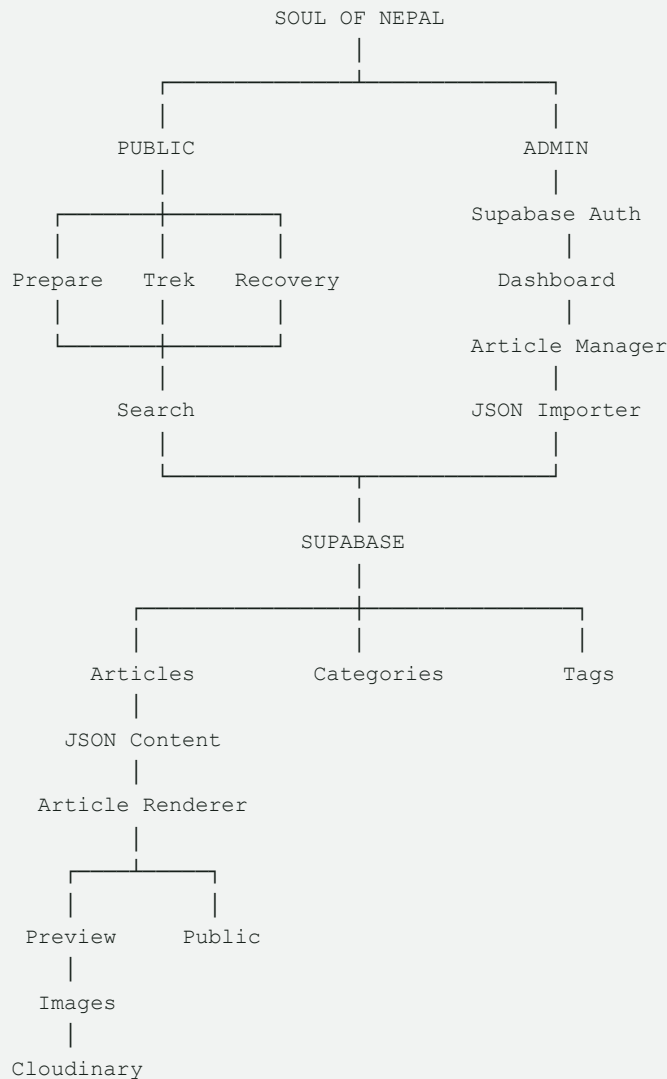
The application should render exactly what the JSON specifies.

3. Technology Stack

Layer	Technology	Purpose
Application	Next.js + TypeScript	Full-stack application, routing and rendering
Routing	Next.js App Router	Application pages and dynamic content routes
Database	Supabase PostgreSQL	Articles, categories, tags, media, advertisements and settings
Authentication	Supabase Auth	Administrator authentication
Image management	Cloudinary	Image uploads, hosting, optimization and transformations

Layer	Technology	Purpose
Content architecture	Structured JSON	Deterministic article structure and rendering

4. High-Level Architecture



5. Content Architecture

The website has three primary content pillars.

5.1 Prepare for Nepal

Information travelers need before arriving in Nepal.

- Passport requirements
- Visa information

- Travel documents
- Money and currency
- SIM and eSIM
- Internet
- Transportation
- Airports
- Travel insurance
- Safety
- Weather
- Packing
- Cultural etiquette
- Food
- Travel costs
- First-time Nepal guides

5.2 Trekking & Adventure in Nepal

- Trekking routes
- Everest Base Camp
- Annapurna Circuit
- Annapurna Base Camp
- Mardi Himal
- Langtang
- Manaslu
- Mustang
- Trekking permits
- Trekking seasons
- Difficulty
- Altitude
- Acclimatization
- Trekking equipment
- Guides and porters
- Transportation
- Mountain safety

5.3 Recovery & Healing in Nepal

- Recovery after trekking

- Yoga
- Meditation
- Breathwork
- Ayurveda
- Massage
- Relaxation
- Sleep
- Wellness
- Recovery practices
- Wellness destinations
- Yoga experiences
- Spiritual and wellness experiences

The content should primarily remain informational and useful rather than aggressively promotional.

5.4 About Soul of Nepal

The About page should explain:

- What Soul of Nepal is
- Why it exists
- Its editorial philosophy
- Its approach to Nepal information
- Contact information

6. Homepage

The homepage is a **content discovery hub**. It should dynamically pull content from Supabase.

HOME

Hero / Introduction

[Search Nepal...]

PREPARE FOR NEPAL

[Article] [Article] [Article]

TREKKING & ADVENTURE

[Article] [Article] [Article]

RECOVERY & HEALING

[Article] [Article] [Article]

Featured Guides

Latest Articles

Popular Articles

Footer

The final visual design is intentionally deferred to the later UI/UX phase.

7. Global Search

The homepage must contain a global search function.

Examples:

```
passport
Mardi Himal
altitude
yoga
```

Search should consider:

- Article title
- Article excerpt
- Article content
- Category
- Tags
- SEO keywords where applicable

Initial Search Technology

Use PostgreSQL full-text search through Supabase initially.

Do not introduce an external search service unless scale or search quality later justifies it.

8. JSON-First Article System

This is the most important component of Soul of Nepal.

The administrator should not manually build articles block-by-block. Instead, the article is prepared externally and converted into the Soul of Nepal JSON structure.

ARTICLE → SOUL OF NEPAL JSON → VALIDATE → PREVIEW → IMAGES → PUBLISH

No AI Interpretation During Rendering

The website itself does not need an AI model to determine what the article structure means.

The JSON explicitly defines it.

For example:

```
{
  "type": "heading",
  "level": 2,
  "text": "Spring Trekking Season"
}
```

This means the renderer should output an H2.

```
{
  "type": "paragraph",
  "text": "Spring is one of the best seasons..."
}
```

This means the renderer should output a paragraph.

The rendering process must be deterministic.

9. Soul of Nepal Article JSON Schema

Create and maintain a strict schema for the article format.

```
{
  "title": "Best Time to Trek in Nepal",
  "slug": "best-time-to-trek-in-nepal",
  "category": "trekking-adventure",
  "excerpt": "A practical guide to choosing the best trekking season in Nepal.",
  "author": "Soul of Nepal",
  "tags": [
    "trekking",
    "nepal",
    "travel"
  ],
  "blocks": [
    {
      "type": "paragraph",
      "text": "Nepal offers some of the world's most spectacular trekking routes."
    },
    {
      "type": "heading",
      "level": 2,
      "text": "Spring Trekking Season"
    },
    {
      "type": "paragraph",
      "text": "Spring is one of the most popular times to trek in Nepal."
    },
    {
      "type": "heading",
      "level": 3,
      "text": "Why Visit in Spring?"
    },
    {
      "type": "list",
      "style": "bullet",
      "items": [
        "Stable weather",
        "Clear mountain views",
        "Rhododendron forests"
      ]
    }
  ]
}
```

The schema should be versionable so future article capabilities can be added without breaking existing articles.

10. Supported Content Blocks

The initial renderer must support at least:

- Paragraph
- Heading
- Bullet list
- Numbered list

- Quote
- Image

Paragraph

```
{  
  "type": "paragraph",  
  "text": "Example paragraph."  
}
```

Heading

```
{  
  "type": "heading",  
  "level": 2,  
  "text": "Example Heading"  
}
```

Supported heading levels:

- H2
- H3
- H4

The article title itself is H1. Avoid arbitrary H1 blocks inside article content.

Bullet List

```
{  
  "type": "list",  
  "style": "bullet",  
  "items": [  
    "First item",  
    "Second item",  
    "Third item"  
  ]  
}
```

Numbered List

```
{
  "type": "list",
  "style": "numbered",
  "items": [
    "First step",
    "Second step",
    "Third step"
  ]
}
```

Quote

```
{
  "type": "quote",
  "text": "Example quotation."
}
```

Image

```
{
  "type": "image",
  "src": "CLOUDINARY_URL",
  "alt": "Trekking walking through the Himalayas",
  "title": "Trekking in Nepal",
  "caption": "Trekking approaching a Himalayan trail.",
  "credit": "Soul of Nepal",
  "alignment": "center"
}
```

11. Inline Formatting

The article system must support:

- **Bold**
- *Italic*
- Links

Prefer structured inline content rather than arbitrary raw HTML.

```
{
  "type": "paragraph",
  "content": [
    {
      "text": "Nepal is "
    },
    {
      "text": "beautiful",
      "bold": true
    },
    {
      "text": " throughout the year."
    }
  ]
}
```

Link example:

```
{
  "text": "travel information",
  "link": "https://example.com"
}
```

All links and rich content must be safely sanitized before rendering.

12. JSON Importer

The admin panel must contain a dedicated article import interface.

IMPORT ARTICLE

Paste Soul of Nepal JSON here...

[Validate JSON]

Validation

When the administrator clicks **Validate JSON**, the complete object must be checked against the Soul of Nepal schema.

Valid result:

✓ Valid Soul of Nepal Article

[Generate Preview]

Invalid result:

```
Invalid Article JSON

Location:
blocks[4]

Problem:
"type" is not recognized.

Allowed values:
paragraph
heading
list
quote
image
```

Errors must identify the problem as precisely as practical.

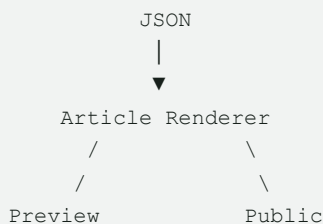
13. Preview System

After successful validation, the administrator clicks:

```
[ Generate Preview ]
```

The article should immediately render as it will appear on the website.

Critical architecture rule: Use the same Article Renderer for both preview and the public article. Do not create a separate fake preview renderer.



This guarantees that the preview and published article use the same rendering logic.

14. Direct Editing From Preview

After generating the preview, the administrator should be able to make final adjustments directly from the rendered article.

Hovering between content blocks should reveal a + control.

BEST TIME TO TREK IN NEPAL

Nepal offers some of the world's...

[+]

SPRING TREKKING SEASON

Spring is one of the most popular...

[+]

WHY VISIT IN SPRING?

...

Clicking the plus control should provide:

ADD CONTENT

[Image]

[Paragraph]

[Heading]

[List]

[Quote]

The primary Phase 1 use case for this control is image insertion.

15. Image Workflow

The administrator should normally complete the article text first. Images are then added during the preview/finalization stage.

JSON → PREVIEW → CHECK TEXT → INSERT IMAGES → SAVE DRAFT → PUBLISH

Image Insertion Interface

```
ADD IMAGE

[ Upload Image ]

Alt Text
[ _____ ]

Image Title
[ _____ ]

Caption
[ _____ ]

Credit
[ _____ ]

Alignment
[ Left | Center | Right ]

[ Upload & Insert ]
```

After insertion, the image becomes part of the article's structured data.

16. Cloudinary Architecture

Cloudinary is the dedicated image handling system.

```
Administrator
  |
  ▼
Select Image
  |
  ▼
Secure Upload
  |
  ▼
Clouldinary
  |
  ▼
Asset URL + Metadata
  |
  ▼
Image Block
  |
  ▼
Article
```

Use Clouldinary for:

- Image hosting
- Optimization

- Transformations
- Responsive image delivery
- Resizing
- Format optimization

Cloudinary Security

Never expose the Cloudinary API secret to client-side JavaScript.

```
CLOUDINARY_CLOUD_NAME  
CLOUDINARY_API_KEY  
CLOUDINARY_API_SECRET
```

Use a secure signed/server-side upload architecture.

The coding agent should request Cloudinary credentials only when it reaches the Cloudinary implementation phase.

17. Database Architecture

Use a **relational + JSON hybrid architecture**.

Do not store the entire website as one giant HTML field.

Recommended core tables:

```
articles  
categories  
tags  
article_tags  
media  
ad_slots  
ad_configurations  
site_settings
```

Article structure can be stored as validated JSON while important metadata remains relational.

This provides:

- Flexible content
- Reliable categories
- Tags
- Search
- Filtering
- Publishing control
- Media management

- Future scalability

17.1 Articles

```
id
title
slug
excerpt
category_id
content_json
featured_image_id
author_id
status
published_at
created_at
updated_at
seo_title
seo_description
canonical_url
is_featured
view_count
```

17.2 Categories

```
id
name
slug
description
created_at
updated_at
```

Initial categories:

```
prepare-for-nepal
trekking-adventure
recovery-healing
```

17.3 Tags

```
id
name
slug
created_at
```

17.4 Article Tags

```
article_id
tag_id
```

Add an appropriate unique constraint to prevent duplicate relationships.

17.5 Media

```
id
cloudinary_public_id
secure_url
width
height
format
alt_text
title
caption
credit
created_at
updated_at
```

18. DB.sql Requirement

MANDATORY: Create a canonical database definition file inside the repository.

Recommended location:

```
/database/DB.sql
```

This file must contain the database definition required to reproduce the database architecture.

It should cover:

- Tables
- Columns
- Data types
- Primary keys
- Foreign keys
- Indexes
- Constraints
- Functions
- Triggers
- RLS policies
- Relevant database configuration

Absolute rule: Whenever the database architecture changes, update `DB.sql` immediately. Never postpone database documentation.

Migration Files

Also maintain proper Supabase migration files.

```
supabase/  
  migrations/  
    001_initial_schema.sql  
    002_article_system.sql  
    003_media.sql  
    004_ads.sql
```

Migration history should remain reproducible and synchronized with the actual database architecture.

19. Authentication & Authorization

Supabase Auth

Use Supabase Auth for administrator authentication.

Do not build a custom password authentication system.

Row Level Security

Use Supabase Row Level Security for database protection.

```
Authentication  
  ↓  
Authorization  
  ↓  
Server-Side Checks  
  ↓  
Supabase RLS
```

Public visitors must never be able to perform administrative mutations.

They must not be able to:

- Create articles
- Modify articles
- Delete articles
- Upload administrative media
- Change advertisements
- Modify site settings

- Modify categories
- Modify tags

20. Admin Panel

Admin Entry

A discreet footer entry can open the administrator login.

```
Footer
```

```
About
```

```
Contact
```

```
Staff
```

For example:

```
Staff
```

```
↓
```

```
Supabase Auth
```

```
↓
```

```
Protected Admin Dashboard
```

The obscure footer link is not a security mechanism. Security must come from authentication, authorization and RLS.

Admin Dashboard

```
SOUL OF NEPAL ADMIN
```

```
Articles
```

```
└─ Manage Articles
```

```
└─ Import Article
```

```
Media
```

```
└─ Media Library
```

```
Categories
```

```
└─ Manage Categories
```

```
Tags
```

```
└─ Manage Tags
```

```
Advertisements
```

```
└─ Ad Settings
```

Keep the first admin dashboard deliberately simple.

Article Management

Admin should be able to:

- Import article
- Create article
- Edit article
- Preview
- Save draft
- Publish
- Unpublish
- Archive
- Delete

Article list should display:

```
Title
Category
Status
Updated
Published
Actions
```

Recommended status values:

```
draft
published
archived
```

21. Publishing Workflow

**WRITE ARTICLE → CONVERT TO JSON → PASTE → VALIDATE → PREVIEW → EDIT → ADD
IMAGES → SAVE DRAFT → FINAL PREVIEW → PUBLISH**

The administrator must be able to perform this entire process without touching source code.

Important UX Principle

The system should eliminate repetitive CMS work.

The administrator should not have to manually create a paragraph, then a heading, then another paragraph, then a list, etc.

The structured JSON should already contain this information.

22. SEO Foundation

Each article should support dedicated SEO metadata.

- SEO title
- Meta description
- Slug
- Canonical URL
- Primary keyword
- Secondary keywords
- Featured image
- Image alt text
- Author
- Published date
- Updated date
- Category
- Tags

Next.js should generate appropriate metadata from article data.

Build the technical SEO foundation including:

- Page metadata
- Canonical URLs
- Open Graph metadata
- Article structured data where appropriate
- Sitemap
- Robots configuration
- Clean article slugs

URL Structure

Use clean, readable article routes.

```
/article/best-time-to-trek-in-nepal
```

Keep slugs lowercase, readable, stable and relevant.

23. Advertising Architecture

Build advertising infrastructure from the beginning, but advertisements must not create empty spaces when no advertisement is available.

Critical behavior: If an ad slot has no active advertisement, it must render nothing and consume no unnecessary visible layout space.

Reusable Ad Slots

```
homepage-top
homepage-between-sections

category-top
category-between-articles

article-top
article-middle
article-bottom
```

The exact placement can be adjusted later.

Conditional Rendering

```
AdSlot
  ↓
Is active ad configured?
  |
  ├── NO → Render nothing
  |
  └── YES → Render advertisement
```

Do not show large blank boxes labeled "Advertisement" when there is no ad.

Ad Database

ad_slots

```
id
name
slug
description
location
is_active
created_at
updated_at
```


ad_configurations

```
id
slot_id
provider
configuration
is_active
created_at
updated_at
```

Do not tightly couple the article renderer to a single advertising provider. Use an abstraction that allows an approved provider to be connected later.

Ad Placement Philosophy

Prioritize:

1. Readability
2. User experience
3. Content
4. Monetization

The infrastructure can support multiple placements without forcing every placement to be active.

24. Security Requirements

Security is not optional. The system should be designed securely from the beginning rather than retrofitted after launch.

Never

- Expose the Supabase service-role key to the browser.
- Expose the Cloudinary API secret to client-side JavaScript.
- Commit real credentials to Git.
- Trust client-side authorization.
- Blindly render arbitrary unsanitized HTML.
- Allow unauthenticated users to perform admin mutations.
- Use an obscure admin URL as the security boundary.
- Log secrets.
- Commit real `.env` secrets.

Always

- Use environment variables.
- Validate JSON server-side.
- Use Supabase Auth.
- Use Row Level Security.
- Perform server-side authorization checks.
- Validate uploaded files.
- Sanitize rich content and links.
- Protect administrative mutations.

25. Environment & Credentials

Potential environment variables:

```
NEXT_PUBLIC_SITE_URL

NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY

SUPABASE_SERVICE_ROLE_KEY

CLOUDINARY_CLOUD_NAME
CLOUDINARY_API_KEY
CLOUDINARY_API_SECRET
```

Only variables genuinely safe for browser exposure should use the `NEXT_PUBLIC_` prefix.

Credential Collection Rule

The coding agent should ask for credentials only when the relevant integration is being implemented.

Before asking, check whether the credential already exists in the environment.

Never print secret values back to the user.

26. Recommended Project Structure

```
src/  
  app/  
    page.tsx  
  
    prepare-for-nepal/  
    trekking-adventure/  
    recovery-healing/  
  
    article/  
  
    about/  
  
    staff/  
      login/  
      dashboard/  
      articles/  
      import/  
      media/  
      categories/  
      tags/  
      ads/  
  
    components/  
      article/  
      admin/  
      ads/  
      search/  
  
    lib/  
      supabase/  
      cloundinary/  
      article/  
      validation/  
      search/  
  
    types/  
      article.ts  
      database.ts  
  
  database/  
    DB.sql  
  
  supabase/  
    migrations/  
  
  public/
```

The exact structure may be adjusted if the agent has a better justified architecture. Do not modify architecture merely for stylistic preference.

27. Phased Execution Plan

PHASE 1 — Project Foundation

- Initialize Next.js
- Initialize TypeScript
- Configure App Router
- Establish environment-variable strategy
- Connect Supabase
- Create basic application structure

Deliverable: Application runs and connects securely.

PHASE 2 — Database

- Create database schema
- Create migrations
- Create DB.sql
- Implement indexes
- Implement constraints
- Implement RLS

Deliverable: Reproducible and secure database.

PHASE 3 — Authentication

- Supabase Auth
- Staff login
- Protected dashboard
- Authorization
- Server-side protection

Deliverable: Only authorized administrators can manage content.

PHASE 4 — JSON Article Engine

- Create JSON schema
- Create TypeScript types
- Create runtime validation
- Create block components
- Create Article Renderer

Deliverable: Valid JSON renders correctly.

PHASE 5 — Import + Preview

- JSON input
- Validation
- Error reporting
- Preview

Deliverable: Administrator can paste JSON and instantly preview an article.

PHASE 6 — Preview Editor

- Direct editing
- Heading editing
- Paragraph editing
- Block insertion
- Inline plus controls

Priority: Image insertion.

PHASE 7 — Cloudinary

- Secure uploads
- Cloudinary integration
- Media metadata
- Image blocks
- Responsive image handling

Deliverable: Administrator can upload and insert images directly into an article.

PHASE 8 — Publishing

- Draft
- Published
- Archived
- Publish
- Unpublish
- Delete

Deliverable: Complete publishing workflow.

PHASE 9 — Categories + Search

- Three primary categories
- Tags
- Global search
- Related content

Deliverable: Visitors can discover content naturally.

PHASE 10 — Homepage

- Search
- Category sections
- Latest articles
- Featured articles
- Popular articles

Deliverable: Homepage dynamically represents the content database.

PHASE 11 — Advertising

- Ad slots
- Ad configuration
- Conditional rendering
- Provider abstraction

Deliverable: Empty slots disappear completely; configured ads can render.

PHASE 12 — SEO

- Metadata
- Canonical URLs
- Open Graph
- Article structured data
- Sitemap
- Robots
- Clean slugs

Deliverable: Strong technical SEO foundation.

PHASE 13 — Testing

Run the complete functional and security acceptance test suite before beginning the final visual design phase.

PHASE 14 — Final UI/UX

Only after the functional platform is stable should the final Soul of Nepal visual identity be implemented.

28. Testing & Acceptance Criteria

Authentication

- ☐ Admin can authenticate through Supabase Auth.
- ☐ Admin can log out.
- ☐ Unauthenticated users cannot access the dashboard.
- ☐ Unauthorized users cannot perform admin mutations.
- ☐ Sessions are handled securely.

JSON

- ☐ Valid JSON is accepted.
- ☐ Invalid JSON is rejected.
- ☐ Missing fields are identified.
- ☐ Invalid block types are identified.
- ☐ Invalid heading levels are identified.
- ☐ Malformed image objects are identified.

Articles

- ☐ Article can be imported.
- ☐ Article can be previewed.
- ☐ Article can be edited.
- ☐ Article can be saved as draft.
- ☐ Article can be published.
- ☐ Article can be unpublished.
- ☐ Article can be archived.
- ☐ Article can be deleted.

Images

- ☐ Image upload works.
- ☐ Cloudinary receives the image.
- ☐ Image URL is stored correctly.
- ☐ Alt text is stored.
- ☐ Image title is stored.

- ❑ Caption is stored.
- ❑ Credit is stored.
- ❑ Image can be inserted between existing blocks.

Search

- ❑ Search finds titles.
- ❑ Search finds relevant content.
- ❑ Search finds tags.
- ❑ Search finds categories.
- ❑ Search works on mobile.

Advertising

- ❑ Empty ad slots render nothing.
- ❑ Disabled ads do not appear.
- ❑ Active ads can render.
- ❑ Multiple slots can be independently configured.

Database

- ❑ Database migrations execute successfully.
- ❑ DB.sql reflects the current architecture.
- ❑ RLS policies work as expected.
- ❑ Public users cannot perform administrative operations.

Production

- ❑ Production build succeeds.
- ❑ No secrets are exposed.
- ❑ No critical console errors exist.
- ❑ Public articles load correctly.
- ❑ Search works.
- ❑ Admin workflow works.

29. Vibe-Coding Agent Rules

These rules are mandatory during the implementation.

- **DO NOT** spend Phase 1 time creating the final visual identity.

- **DO** build the complete functional pipeline.
- **DO NOT** use an AI model merely to interpret article structure during rendering.
- **DO** make the JSON the source of truth.
- **DO NOT** require manual block creation for normal article imports.
- **DO** make JSON import and deterministic rendering the primary workflow.
- **DO** make image insertion possible from the preview.
- **DO NOT** expose credentials in client-side code.
- **DO** use environment variables and secure server-side architecture.
- **DO NOT** create undocumented database changes.
- **DO** update DB.sql whenever the database changes.
- **DO NOT** create blank advertisement boxes when no advertisement exists.
- **DO** make empty ad slots invisible.
- **DO NOT** over-engineer the admin dashboard.
- **DO** keep the initial CMS extremely simple.
- **DO** ask for required credentials at the relevant phase.
- **DO** validate data both client-side and server-side.
- **DO** use the same renderer for preview and published articles.
- **DO** stop at the end of each major phase and report progress.

Phase Reporting Format

PHASE COMPLETED

Implemented:

- ...
- ...
- ...

Tested:

- ...
- ...
- ...

Database changes:

- ...

Files created/modified:

- ...

Credentials required:

- ...

Known issues:

- ...

NEXT PHASE:

- ...

30. Definition of Done

The first functional version is complete only when the administrator can publish an article without touching the source code.

**WRITE ARTICLE → CONVERT TO JSON → PASTE JSON → VALIDATE → PREVIEW → EDIT →
INSERT IMAGES → SAVE DRAFT → FINAL PREVIEW → PUBLISH → SEARCH**

The administrator should be able to:

1. Write an article outside the CMS.
2. Convert the article to Soul of Nepal JSON.
3. Paste the JSON into the admin panel.
4. Validate it.
5. Generate an instant preview.
6. Make final adjustments from the preview.
7. Hover between sections and click +.
8. Upload images to Cloudinary.

9. Add image alt text, title, caption and credit.
10. Insert the image at the desired position.
11. Save the article as a draft.
12. Perform a final preview.
13. Publish it.
14. Open the public article.
15. Find the article through global search.

Final Phase 1 objective: The owner should be able to publish a complete, properly structured Soul of Nepal article in minutes rather than manually constructing a webpage.

Final Product Principle

Soul of Nepal should behave like a lightweight publishing platform built specifically around the owner's workflow.

The administrator writes or obtains the content elsewhere, converts it into the defined JSON structure, imports it, visually checks it, adds photography, and publishes it.

The system exists to **remove repetitive work**, not create more CMS work.

The database, JSON schema, renderer, image system, search, SEO infrastructure, advertising system and admin panel must all support that goal.

JSON → VALIDATE → PREVIEW → EDIT → IMAGES → DRAFT → PUBLISH

VIBE-CODING AGENT — FINAL DIRECTIVE

Build the functional system first. Use skeleton UI. Do not get distracted by visual design. Ask for credentials only when required. Keep the database documented in DB.sql. Keep the application secure. Make the JSON the source of truth. Make article publishing extremely fast and simple.